DAN THE DIGITAL MAN
dtdm.us

PROJECT HOPPING

Using Claude Projects as Interchangeable Personas for Faster, Deeper Idea Development

> *“Don't build ten agents. Build ten lenses, and walk your idea through each one.”*

A free guide from Dan the Digital Man

Why This Exists

If you use Claude AI for real work, you've probably already discovered Projects — the ability to give a chat its own instructions and its own set of reference files. Most people use this the obvious way: one Project per client, one Project per app, one Project as a junk drawer for “misc.”

That's fine. It's also leaving most of the value on the table.

This guide lays out a different way to use Projects: as interchangeable personas you deliberately hop between mid-conversation — or mid-idea — to run the same problem through different lenses, without spinning up separate automated agents for each one. It's a lightweight alternative to “agent-izing” everything, built for the specific case where you, a human, want to keep thinking alongside the AI rather than hand a task off and walk away.

It costs nothing to set up. It uses a feature you already have. And once the pattern clicks, it changes how you use Claude for anything that benefits from more than one point of view — which is most interesting work.

The Core Idea

A Claude Project is really three things bundled together: a system instructions field, a set of uploaded reference files, and a persistent memory of past chats within that Project. Together, those three things function exactly like a distinct AI persona or agent — the same project instructions and files that would normally require setting up a separate automated agent to get a differentiated “point of view.”

A Marketing Project reads your request differently than a Development Project does. A Research Project asks different questions than a Brainstorming Project. Not because Claude itself changed — but because the frame it's operating inside changed.

Project hopping means treating that frame-shift as a deliberate move. Instead of trying to get one mega-Project to be good at everything, you build several narrow, well-defined Projects — each with a clear personality and job — and you consciously walk your idea or your conversation through them in sequence.

The result behaves less like switching browser tabs and more like consulting a different specialist. You're not asking Claude to “now think like a marketer.” You're walking into the room where the marketer already lives.

Why This Beats Reaching for an Agent (Sometimes)

To be clear up front: automated agents absolutely have their place. If you want a process that runs on its own — scraping data on a schedule, triaging inbound leads, monitoring a pipeline, executing a defined workflow without you in the loop — an agent is the right tool. That's what agents are built for: unattended execution.

Project hopping solves a different problem: back-and-forth, human-in-the-loop collaboration where you want to think through something from multiple angles without the overhead of standing up, configuring, and maintaining several separate automated systems just to get a second opinion.

What you get from hopping projects instead:

- No infrastructure. No API keys, no separate configuration, no maintenance — it's a dropdown menu you already have.
- You stay in the loop. You see every phase, every angle, every draft — nothing runs invisibly while you wait.
- Cheap experimentation. Standing up a new lens costs you a text box and ten minutes, not an afternoon of agent config.
- Natural checkpoints. Each hop is a deliberate pause where you, the human, decide what carries forward and what doesn't.
- One idea, many angles — fast. Run a single concept through Development, then Marketing, then Brand, then back to Development, in the time it takes to have one long conversation.

Use agents when you want something to run without you. Use project hopping when you want to think better with an AI that's actually in the room with you — just wearing a different hat each time you need one.

Setting Up Your Projects

1. Name each Project for its function, not its topic

Skip vague names like “Ideas” or “Work Stuff.” Name each Project after the lens it represents: Development, Marketing, Brand & Voice, Research, Brainstorming, Legal/Risk Review, Customer Perspective — whatever roles actually show up in how you build things.

2. Start every Project's instructions with its own name, stated plainly

This is the detail people skip, and it's the one that matters most. Claude does not inherently know which Project it's operating in — the label on the sidebar is not visible inside the chat itself. If the instructions don't state the Project's identity, Claude has to infer its role purely from files and phrasing, which is fragile.

Open every Project's instruction field with a direct, unambiguous statement, for example:

Example opening line — Marketing Project

```
You are operating inside the “Marketing” Project for [Your Brand].
Read every request through a marketing and conversion lens:
audience, positioning, funnel stage, and call-to-action —
even when the request doesn't explicitly mention marketing.
```

Example opening line — Development Project

```
You are operating inside the “Development” Project for [Your Brand].
Read every request through a build-and-ship lens: architecture,
trade-offs, and what's actually deployable — not just what's
theoretically elegant.
```

That one sentence does the work of turning a folder of files into a persona with a stated identity, rather than a persona Claude has to guess at.

3. Load each Project with only what that persona needs

Resist the urge to upload everything everywhere. A Development Project stuffed with brand voice guides and ad copy templates is a Development Project with noise in it. Keep each Project's file set tight and specific to the lens it represents — that's what makes the persona distinct rather than a diluted copy of every other Project.

4. Decide what tools and connectors each persona has access to

If you use integrations — Google Drive, Notion, HubSpot, Canva, GitHub, Supabase, and so on — those can be scoped per Project the same way files and instructions are. A Marketing persona might need Canva and email tools; a Development persona might need GitHub and a database connector; neither necessarily needs the other's. Decide this deliberately when you set the Project up, rather than discovering mid-hop that the wrong persona has access to the wrong system.

The Missing Layer: General Instructions in Your User Profile

Projects give you specialized personas. But some things should be true no matter which persona you're talking to — your name, your communication preferences, formatting rules, how you want files delivered, how you want mistakes handled. That layer doesn't belong inside any one Project. It belongs in your account-level profile settings, sometimes labeled “custom instructions” or “what should Claude know about you.”

Think of it as the personality traits every persona inherits, regardless of which Project you're in — and Project-hopping specifically is one of those traits worth encoding at the profile level, since it governs how you move between Projects in the first place, not what any single Project does.

Below is the actual instruction Dan the Digital Man uses at the profile level to make Claude handle Project switches correctly — feel free to adapt it word-for-word:

Profile-level instruction: Project Switch = Persona Switch

```
Project switch = persona switch. Each Claude Project functions
as a distinct persona/agent — its own instructions, files, and
lens, the same way setting up a separate agent would. When I
state I've switched projects, that is not a location update — it
means every reply going forward must be filtered through that
new project's actual instructions and files, exactly as if a
different agent were now answering.

Filtering through the new project's context does not mean
surfacing, quoting, summarizing, or referencing that project's
instructions or files back to me as if I had typed them into the
prompt myself. The new context shapes tone, scope, priorities,
and default assumptions in the background — it is never
announced, injected as visible text, or treated as something I
need briefed on. Just answer as that persona would.

Do not cling to, re-assert, or defend the prior project's
instructions, labels, or context after a stated switch. Do not
attempt to re-derive or argue about which project is active
based on which tools, files, or knowledge base happen to still
be accessible — that visibility is unreliable and not
authoritative over what I state. If there is a genuine
mismatch between what I describe and what is accessible, note
it once as a limitation, never as a correction, and do not
repeat it once acknowledged.
```

Without this instruction, Claude will often try to be helpful in exactly the wrong way — quoting the new Project's instructions back at you, or arguing about which Project is actually active based on stale context. This instruction heads that off: the switch happens silently, in how Claude reasons and responds, never as a visible announcement.

Running an Idea Through Phases

Once your Projects are named, instructed, and stocked with the right files, the workflow itself is simple: bring your idea to one Project, work it until that lens has given what it can, then deliberately move to the next.

A typical sequence:

1. Brainstorming — loose, wide, no self-editing. Get volume and range before judging anything.
2. Research — stress-test the assumptions. What's already known, what's been tried, what's the competitive landscape.
3. Development — what would it actually take to build this. Architecture, stack, effort, dependencies.
4. Brand & Voice — does this fit who you are publicly. Naming, tone, positioning.
5. Marketing — who is this for, how do they find it, what makes them act.
6. Risk / Legal / Customer Perspective — what could go wrong, who could be upset, what's the liability.

You won't run every idea through every phase, and the order isn't fixed — a technical idea might start in Development and only later need Marketing; a campaign idea might start in Marketing and only later need Development to check feasibility. The point isn't the specific sequence. It's that each hop forces a genuinely different read on the same material, from a persona built to catch what the others miss.

The most-skipped, highest-leverage persona

Most people naturally build a Development persona and a Marketing persona. Almost nobody builds a dedicated adversarial one — a Project whose entire job is to find the reasons an idea fails. That's a mistake, because none of the other personas are built to produce that output. A Marketing persona is instructed to find the angle that sells; a Development persona is instructed to find a way to build it. Neither is instructed to argue against the idea existing at all. A standalone Skeptic or Risk-Review Project, with instructions that explicitly say to look for reasons to kill the idea rather than reasons to like it, catches what every other phase is structurally biased to miss.

Order dependency — which hops actually gate others

Not every phase is optional or reorderable. Some genuinely block others: a Research finding can invalidate what's worth building; a Brand constraint can rule out a Marketing angle; a Risk-Review finding can kill the idea outright. Decide which hops are true prerequisites versus which can happen in any order — or in parallel across separate chats — rather than running the sequence top to bottom by default.

What to say when you hop

Keep the hand-off minimal. Because your Project instructions get loaded fresh on every message (covered in detail two sections from now), the new Project already knows its own name and its own lens the instant you land in it — you don't need to state which Project you've moved into, and you don't need to re-explain the idea. All you're doing is flagging that a switch happened, so Claude doesn't keep running on the old lens by default. The recommended prompt is just:

> *“Project changed, proceed.”*

Worth naming, since you'll be typing it constantly: PCP — short for “project changed, proceed.” Same three words, every time, regardless of which Project you've moved into — the Project's own opening line does the identifying, not your prompt.

The One Way This Strategy Defeats Itself

The entire value of project hopping is getting a genuinely different read on the same material from each persona. That value disappears if each new persona just politely agrees with whatever the last one concluded — at that point you're not getting multiple lenses, you're getting one lens with extra steps and a nicer-sounding sense of thoroughness.

This is a real risk, not a hypothetical one. Left to its own devices, an AI tends to build on what came before rather than push back against it — which is exactly the wrong default for a workflow whose entire point is disagreement between phases.

The fix is to prompt for it explicitly at each hop, rather than assuming friction will surface on its own:

> *“Don't just build on the previous phase's conclusions — actively critique them from this Project's perspective. What would this lens flag as a problem that the last one missed or waved past?”*

Treat agreement between phases as a fact worth double-checking, not a sign the idea is solid. If every persona independently likes the idea for genuinely different reasons, that's real signal. If every persona likes it because each one quietly deferred to the last, that's just one opinion wearing several hats.

How Context Loading Actually Works

A quick, practical note on the mechanics behind all of this — because it changes how you should think about cost and timing.

Project instructions: loaded every message, always

Your Project's instructions are short, so they get read into every single message you send inside that Project. Not once per chat — every message. There's no moment where they “wear off” partway through a conversation.

Project files: loaded every message too — just not always in full

This is the part people assume works like a one-time upload, and it doesn't. Every message you send, Claude re-checks your Project's files for what's relevant to that specific message:

- Small file set: everything gets read in fully, every message — same as instructions.
- Large file set (most of Dan's Projects): Claude automatically switches to a search-and-retrieve mode, pulling just the relevant pieces of the relevant files for that message, rather than the whole library every time.

Either way, the point is the same: this isn't a “load once when the chat opens” system. It's a fresh check on every single message you send.

New chat, same Project — what carries over and what doesn't

Opening a new chat inside a Project always gives you that Project's instructions and files again, automatically. What it does not give you is the conversation history from a different chat in that same Project — that's a separate memory feature, not part of the Project's knowledge base. If something from an earlier chat needs to carry forward, it has to be in the Project's files, in memory, or in a carryover prompt like the one covered next.

Memory doesn't cross Projects either

This matters more once you're deliberately hopping than it does in normal use. Each Project keeps its own separate memory — whatever Claude has picked up about your preferences and prior conversations in the Development Project stays in the Development Project. It doesn't follow you into Marketing, Brand, or anywhere else. Every persona genuinely starts fresh on that front, which is part of what makes each one an honest, independent lens rather than a costume over one continuous memory — but it also means nothing about a decision made in one Project is visible to another unless you explicitly carry it over yourself.

Why this matters for cost

Because Project instructions and files get re-checked on every message, not just once, a heavily-loaded Project has a real per-message cost — not just a one-time cost at the start of a session. That's part of why the carryover-prompt habit below is worth building: it's not just about keeping conversations manageable, it's about not paying that re-check cost forever inside one sprawling chat.

Managing Long Chats and Credits

One honest trade-off: long, multi-phase conversations accumulate context, and context costs credits every time it's re-processed. A chat that's been through five Project hops and forty exchanges is an expensive chat to keep extending indefinitely, in the same way a very long agent run burns more compute the longer it goes.

The fix isn't to avoid long conversations — it's to hand off deliberately instead of letting a chat run until it's unwieldy. When a conversation has done its job in one Project, or is simply getting long, ask the AI itself for a carryover prompt: a compact summary written specifically to seed the next chat, whether that next chat stays in the same Project or hops to another.

Carryover prompt — what to ask for

```
Summarize this conversation into a carryover prompt I can paste
into a new chat. Include: the core idea, key decisions made,
open questions, and anything the next conversation needs to
know to pick up where this one left off. Write it as a prompt
I would send, not as a report to me.
```

This does two things. It keeps each individual chat leaner and cheaper to run, and it forces a useful discipline: naming, explicitly, what actually matters enough to carry forward versus what was just part of getting there. Not everything from a brainstorming session needs to survive into the development phase — the carryover prompt is where you decide what does.

Paste that carryover prompt into the next chat — same Project or a different one — and you're back up to speed in one message instead of re-explaining an hour of work.

Consolidate what you learn somewhere central

Because memory doesn't cross Projects, insights from a hop-heavy workflow end up scattered across several independent personas with no shared record between them. Left alone, the synthesis of “what did we actually learn across all six phases” only ever exists in your head — which defeats some of the value of having run the process at all.

Worth building the habit of logging each phase's real conclusions somewhere central: a running document, a spreadsheet, or a dedicated “Master Reference” Project whose entire job is holding the synthesized output of every other Project. That last option keeps the whole thing inside Claude — a Project you periodically feed the key findings from each hop, that you can then query later without having to reconstruct the sequence from memory.

Three Worked Examples

Putting all of the above together, here's how a full workflow actually looks in practice — from a loose idea to a finished deliverable — across three different kinds of work. Each example assumes you've already set up the profile-level system instructions covered earlier (Project Switch = Persona Switch), so every hop below is a clean handoff rather than something you have to re-explain each time.

Each example also uses a Master Project — the consolidation habit from the previous section, made concrete. Think of it as home base: a Project that doesn't do the specialized work itself, but holds the running synthesis — the current framework, the key decisions, the state of the idea — as it passes through every other Project. You feed it summaries as you go and can return to it any time to see the whole shape of the thing, rather than reconstructing it from memory across five different chats.

Two independent moves, not one

Before the examples: switching Projects and starting a new chat are not the same move, and you can combine them independently. Starting a new chat inside the same Project keeps the lens but sheds accumulated conversation weight — good for cost and focus when a phase is running long but isn't done. Switching Projects changes the lens itself, carried over via a handoff prompt — good when the work itself has moved into a different kind of problem.

Because each Project is its own persistent thread, you can also run more than one in parallel: a Development Project actively debugging in one chat while a separate Marketing Project independently drafts launch copy in another, each making progress on its own timeline. Sometimes those threads run fully independently for a while. Sometimes they need to touch — a Development constraint changes what Marketing can promise, or a Marketing deadline changes what Development has to prioritize — and that's the moment for a short carryover prompt to cross-pollinate, not a full merge of the two threads.

Example One: Building a Coding Product

Idea in question: a small SaaS tool — say, an API-based habit-tracking service. Master Project holds the running spec and decision log throughout.

The Project lineup

- Brainstorm — wide-open concept exploration, no self-editing.
- Research — competitive landscape, technical feasibility, prior art.
- Architecture — stack decisions, system design, trade-offs.
- Build & Debug — implementation, running the Debug Protocol on issues as they surface.
- QA & Ship — testing, deployment checklist, launch readiness.

Idea → framework → working model → deliverable

Loose in Brainstorm, then Research validates it before engineering time gets spent. Architecture turns it into a real system design — data model, stack, API surface. Build & Debug is the longest phase: real code, iterating. QA & Ship is the deliverable: something actually deployable.

Example opening line — Architecture Project

```
You are operating inside the "Architecture" Project for [Your
Product]. Read every request through a system-design lens:
data model, stack choice, and trade-offs — flag anything
that's elegant but not actually buildable on the current stack.
```

Example opening line — Build & Debug Project

```
You are operating inside the "Build & Debug" Project for
[Your Product]. Full DEBUG PROTOCOL applies to every reported
issue. Deliver whole files, not diffs. State the destination
path above every file.
```

Files: Architecture keeps the system design doc, competitor teardown notes, stack-choice rationale. Build & Debug keeps the repo structure/README, recent error logs, coding style guide.

Phase vs. lens switch: Move Brainstorm → Research → Architecture → Build as feasibility and design firm up. But if Build & Debug turns up an architecture-level flaw, hop back to Architecture rather than patching around it in Build.

Handoff moment: Mid-build, after solving a gnarly bug through dead-end attempts — get a carryover prompt with the final solution and file state, then start fresh in the same Build & Debug Project.

Example Two: Marketing a Digital Product

Idea in question: a paid ebook or course launch. Master Project holds the running brand voice, positioning decisions, and funnel notes.

The Project lineup

- Brainstorm — offer ideas, angles, possible hooks.
- Brand & Voice — does the offer fit who you are publicly; naming, tone.
- Marketing & Funnel — audience, channels, launch copy, calls to action.
- Content Development — the actual product: outline, drafting, structure.
- Risk & Skeptic — pricing sanity check, compliance, "why would this fail."

Idea → framework → working model → deliverable

Brainstorm generates offers with no filtering. Brand & Voice checks the winner actually fits before content gets written. Once positioning is settled, run two in parallel: Content Development builds the product while Marketing & Funnel drafts the launch sequence. Before shipping, Risk & Skeptic pressure-tests pricing, claims, and compliance.

Example opening line — Brand & Voice Project

```
You are operating inside the "Brand & Voice" Project for [Your
Brand]. Every idea gets checked against brand fit before
anything else: does this sound like us, does the tone match,
does the name embed anything unintended.
```

Example opening line — Marketing & Funnel Project

```
You are operating inside the "Marketing & Funnel" Project for
[Your Brand]. Read every request through audience, positioning,
funnel stage, and call-to-action — flag any gap between
traffic and conversion before recommending more traffic.
```

Files: Brand & Voice keeps the brand style guide, past copy examples, forbidden-words list. Marketing & Funnel keeps funnel structure notes, past conversion data, channel formatting rules.

Phase vs. lens switch: Move Brainstorm → Brand & Voice → the parallel Content/Marketing split → Risk & Skeptic before launch. If Marketing & Funnel keeps producing copy Brand & Voice would reject, round-trip the two before drafting further.

Handoff moment: Partway through a long draft — say, ten chapters in — get a carryover prompt with the voice, structure, and key points, then start fresh in the same Content Development Project.

Example Three: Developing an Abstract Framework

Idea in question: a new conceptual lens — a way of analyzing systems, a philosophical model, a personal framework (this is, in fact, exactly how the bridle-and-cosmimicry thread earlier in this document actually unfolded). Master Project holds the framework document as it firms up.

The Project lineup

- Exploration — free association, analogies, no rigor yet.
- Research & Grounding — check the idea against existing theory; find overlap and conflict.
- Formalization — turn the loose insight into defined terms, structure, boundary conditions.
- Application & Testing — run the framework against real cases to see if it holds.
- Skeptic — actively try to break it; find the counterexamples.

Idea → framework → working model → deliverable

Exploration is loose analogical thinking, no structure yet. Research & Grounding checks it against existing theory. Formalization gives it real shape: named terms, operators, boundaries. Application & Testing runs it against real cases. For abstract work, the "deliverable" is often the stable, well-tested document itself — what the Master Project has been accumulating throughout.

Example opening line — Formalization Project

```
You are operating inside the "Formalization" Project. Take
loose or intuitive material and give it structure: named terms,
explicit boundary conditions, and how it relates to or departs
from existing frameworks. Precision over elegance.
```

Example opening line — Skeptic Project

```
You are operating inside the "Skeptic" Project. Your job is to
find where a framework breaks: counterexamples, edge cases,
internal contradictions. Do not soften findings to be
encouraging — a flaw not found here ships into the deliverable.
```

Files: Formalization keeps a terminology compendium, references to related frameworks, a structure template. Skeptic keeps known failure modes of similar frameworks, standard philosophical counterarguments.

Phase vs. lens switch: Move Exploration → Research & Grounding → Formalization → Application & Testing as structure firms up. If Skeptic finds a fatal flaw, hop back to Formalization to patch it — not back to Exploration.

Handoff moment: Right after the conversation produces a genuinely new synthesis — capture it clean into a fresh chat in the Formalization Project immediately, rather than letting the same chat keep circling.

The Short Version

- Projects aren't just folders — they're personas, and instructions + files are what give a persona its point of view.
- Name each Project's identity explicitly inside its own instructions; Claude can't see its own sidebar label.
- Scope tools and connectors per Project too, the same way you scope files — decide it on purpose, not by accident.
- Put cross-Project preferences — including how to handle Project switches — in your account-level profile, not in any one Project.
- Hop deliberately between Projects to run one idea through multiple honest lenses, instead of debating internally what a marketer, a developer, or a skeptic would say.
- Build a dedicated Skeptic or Risk-Review persona — it's the one most people skip and the one none of the others are built to replace.
- Prompt each hop to actively critique the last phase's conclusions, not just build on them — agreement between personas should be evidence, not a default.
- Since instructions self-identify and reload every message, the hand-off is just PCP — short for “project changed, proceed.” No need to name the destination or restate the idea.
- Memory doesn't cross Projects, so consolidate what each phase actually concludes somewhere central — a doc, or a dedicated Master Reference Project.
- Agents are for unattended, automated work. Project hopping is for you, thinking, with the AI actually in the room.
- When a chat gets long or a phase is done, ask for a carryover prompt instead of letting the conversation run indefinitely.